

Agentproof: A Real-World Base-Rate Study of Static Verification for Agent Workflow Graphs

Anonymous submission

Abstract

Agent frameworks increasingly encode tool-using behavior as explicit workflow graphs whose topology can be analyzed before deployment—yet whether such graphs actually contain the structural defects static verification targets has never been measured. We build AGENTPROOF, which automatically extracts a unified abstract graph model from four agent frameworks (LangGraph, CrewAI, AutoGen, Google ADK) with no hand-written model, applies six structural checks with witness traces, and compiles a temporal policy DSL (a safety fragment of LTL) to DFAs evaluated both statically, via a graph $\times$ DFA product, and at runtime. Used as a measurement instrument, it enables the first base-rate study of topology defects in real agent workflows: 930 mined from 253 GitHub repositories across all four frameworks, a 120-workflow sample validated against independent LLM-reconstructed ground-truth graphs and adversarial triage. The result is negative: real workflows are tiny (median two non-sentinel nodes) static pipelines, and of 187 flagged defects only two are genuine—43% are extraction artifacts, 50% intentional design. We diagnose extraction fidelity (edge recall 0.64 on real code versus near-perfect on stubs, and as low as 0.37 for implicit-topology frameworks like ADK), not the check algorithms, as the binding constraint, and give the static machinery a defect-independent job: the product construction proves which runtime monitors can never fire, so they can be soundly pruned.

Introduction

Large language models are increasingly deployed as autonomous agents that call external tools and APIs (Schick et al. 2023; Yao et al. 2023). Several orchestration frameworks structure this behavior as explicit *workflow graphs*: LangGraph (LangChain 2025), CrewAI (CrewAI 2025), AutoGen (Wu et al. 2024), and Google ADK (Google 2025) each represent computation steps as nodes and allowable transitions as edges. Yet safety enforcement in these systems remains almost entirely a *runtime* concern: tools such as NeMo Guardrails (NVIDIA 2023) and LlamaGuard (Inan et al. 2023) filter individual calls at execution time, detecting a violation only when the offending path is exercised.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Some defects, however, are topological. Consider an email-triage agent whose developer adds a `draft_response` node but forgets to connect it to `send`: normal-priority emails silently halt at a dead end. A content-level runtime guardrail cannot catch this unless that path runs, but a static analysis of the graph flags the dead end and emits a witness trace. In principle classical model checkers (Clarke, Grumberg, and Peled 1999; Holzmann 1997; Cimatti et al. 2002) verify such properties, but they require hand-translating the system into Promela or SMV—prohibitive for developers iterating on workflows.

This raises a question nobody has answered empirically: *do real agent workflows actually contain the structural defects that static verification targets?* We build AGENTPROOF to find out. It extracts a unified graph model directly from four frameworks’ APIs (no manual modeling), runs six structural checks with witness traces, and compiles a temporal policy DSL to DFAs checked statically via a graph $\times$ DFA product and at runtime. We then use it as a *measurement instrument* on 930 workflows mined from 253 public GitHub repositories.

The answer is deflationary, and that is the point. Topology defects are rare; what limits static verification in practice is not the check algorithms but *extraction fidelity*, which we quantify and which turns out to be strongly framework-dependent. The static machinery still earns a defect-independent job: proving which runtime monitors can never fire, so they can be soundly pruned.

Contributions.

- **The first real-world base-rate study** of structural (topology) defects in agent workflows: 930 workflows from 253 public GitHub repositories across all four frameworks, with a 120-workflow subsample validated against independent LLM-reconstructed ground-truth graphs and adversarial triage (Section). Because nobody had measured it, the rarity is itself the contribution: the validated topology-defect rate is $\approx 0\%$ (2 of 187 flags genuine, 95% CI [0.3, 3.8]%; 43% extraction artifacts, 50% intentional design), and real workflows are mostly small static pipelines (median two non-sentinel nodes).
- **AGENTPROOF**, the measurement instrument: automatic extraction of a unified graph model from four heterogeneous frameworks with no hand-written formal model, six

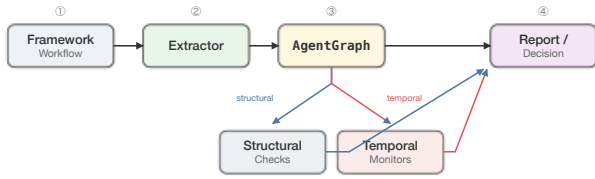


Figure 1: Agentproof extracts a unified graph model from four frameworks, runs structural checks and DFA-compiled temporal policies statically (and at runtime), and emits witness traces.

structural checks with witness traces, and a seven-form temporal DSL over the safety fragment of LTL compiled to DFAs and evaluated statically and at runtime (Section).

- **A quantified diagnosis:** extraction fidelity, not the checks, is the binding constraint. Edge recall drops to 0.64 on real code (0.37 on ADK, 0.77 on AutoGen) and node-kind accuracy to 0.83, both near-perfect on stubs; every triaged structural flag is an extraction artifact (0 of 72; 95% CI $[0, 5.1]\%$), and such flags arise only on LangGraph because the other extractors impose connected topologies by construction.
- **A defect-independent job** for the static machinery: the graph $\times$ DFA product proves which policies can never reach a violation, so their runtime monitors are probably inert and can be pruned (a mean 14 of 15 per workflow; Section), positioning static verification as a sound monitor-selection layer that bridges to leaner runtime shields.

The Agentproof System

Unified graph model. Agentproof represents a workflow as a directed graph $G = (V, E, v_0, V_f)$ with a distinguished entry v_0 and exit set V_f . Each node carries a kind $\kappa_V(v) \in \{\text{ENTRY, EXIT, TOOL, LLM, ROUTER, HUMAN, SUBGRAPH}\}$ and, for tool nodes, a set of bound tool names; each edge is DIRECT, CONDITIONAL, PARALLEL, or LOOP. Framework-specific extractors bridge four heterogeneous representations—each with different entry/exit conventions, edge semantics, and hierarchy models—into this single typed model, eliminating the manual translation that SPIN or NuSMV require. Extraction has two paths: a runtime extractor that reads the compiled graph object, and a static AST fallback (used for large-scale mining, where installing every repository’s dependencies is infeasible).

Structural checks. Six checks run over the topology, each in $O(|V| + |E|)$ and each emitting a *witness trace* on failure. *Exit reachability* verifies $V_f \subseteq \text{Reach}(v_0)$; *reverse reachability* flags livelock nodes reachable from v_0 that cannot reach any exit; *dead-end detection* flags non-exit nodes with no successor; *router shape* requires router outgoing edges to be conditional; *human-gate coverage* checks that no path from v_0 to a sensitive tool avoids every HUMAN node; and *tool declaration* flags tool nodes with an empty tool set. Wit-

ness traces follow standard model-checking practice (Clarke, Grumberg, and Peled 1999) and pinpoint the offending path.

Temporal policies. Beyond topology, Agentproof supports a policy DSL covering the safety fragment of LTL (Pnueli 1977), compiled to deterministic finite automata. The DSL has seven forms: forbidden ($G \neg a$), implication-future ($a \rightarrow F b$), until, bounded response ($a \rightarrow F [\leq k] b$), response chain, conjunction, and disjunction. Each node emits an event symbol $\sigma(v)$ from its kind and tool bindings; base patterns compile to 2–3 state DFAs and bounded response to $k+2$ states (full grammar in the supplementary material).

Static temporal verification. Policies are checked *statically* via a graph $\times$ DFA product: BFS from (v_0, q_0) advances the DFA at each node and follows graph edges, and a reachable product state in the DFA violation set witnesses a violation. Because the product explores a superset of a workflow’s runtime paths, if no violation state is reachable the property holds on *every* runtime execution—a sound guarantee we exploit in Section . Explored states are bounded by $|V| \cdot |Q|$, so verification is linear in graph size for fixed-size DFAs; the same monitors also run at runtime over an event stream with $O(1)$ per-event overhead.

Real-World Base-Rate Study

To estimate defect *base rates* on independently authored workflows—and to measure extractor fidelity on code we did not write—we mined public GitHub repositories across all four frameworks.

Mining. Using GitHub code search we streamed each candidate repository through clone, extraction, and cleanup, yielding 930 workflow graphs from 253 distinct repositories (LangGraph 402, AutoGen 361, CrewAI 114, ADK 53). Extraction used the static AST fallback. Mined workflows are small: a median of 2 non-sentinel nodes (mean 3.9), predominantly static linear pipelines.

Validation. On a stratified 120-workflow sample (40 LangGraph, 20 CrewAI, 35 AutoGen, 25 ADK) we ran two independent LLM-agent passes. First, an agent reconstructed a ground-truth graph from source alone (blind to the extractor, following its node-id conventions), against which we scored node/edge precision–recall and node-kind accuracy. Second, for each of the 187 flagged defects one agent labelled it REAL, EXTRACTION-ARTIFACT, or INTENTIONAL from source, and an adversarial verifier attempted to overturn each label (disagreements recorded as ARGUABLE). Each validated workflow received one ground-truth reconstruction, one triage pass, and one adversarial verification pass (the verifier overturned 4.6% of primary labels). All structural and temporal analyses run in pure Python 3.12 on a commodity laptop (no GPU); the ground-truth and triage agents were Claude Opus 4.8 orchestrated by a multi-agent workflow harness, and all agent outputs are released. We report Wilson 95% confidence intervals for proportions and bootstrap intervals (10,000 resamples over workflows) for fidelity means.

Extraction fidelity is the bottleneck. Node detection stays strong (overall $P=0.91$, $R=0.85$), but edge recall falls to

Table 1: AST-extractor fidelity on real workflows ($n=120$). Perfect on stubs; on real code fidelity is lower and strongly framework-dependent.

Framework	n	Edge P	Edge R	Kind acc
LangGraph	40	0.956	0.682	0.647
CrewAI	20	0.700	0.692	0.929
AutoGen	35	0.957	0.770	0.900
ADK	25	0.403	0.367	0.943
Overall	120	0.798	0.644	0.829

Table 2: Adversarial triage of 187 flagged defects. Every structural flag is an extraction artifact; the human-gate flag is predominantly intentional.

Check	Real	Artifact	Intent.	Arg.
exit_reachability	0	14	0	0
reverse_reachability	0	25	0	0
dead_ends	0	24	0	1
router_shape	0	3	0	0
tool_declarations	0	5	0	0
human_presence	2	9	93	11
Total	2	80	93	12

0.644 (95% CI [0.57, 0.71]) and node-kind accuracy to 0.829, ranging from 0.37 [0.20, 0.55] edge recall on ADK to 0.77 [0.68, 0.86] on AutoGen (Table 1); the small per-framework samples give wide intervals, but ADK is the worst case even at its upper bound. The frameworks that encode topology *implicitly*—ADK’s nested `SequentialAgent/ParallelAgent` composition, AutoGen’s participant lists with dynamic speaker selection—are hardest: the extractor must guess an edge set the source never states, and for ADK it is wrong more often than right. On LangGraph, whose edges are explicit, the dominant error is instead node-kind (`tool→llm`: nodes invoking tools in their body with no declared binding).

The fallback, not the ceiling. This 0.64 is a property of the static AST fallback used for scale, not of the runtime extractors, which read the compiled framework object and are faithful by construction. On the (trusted) runnable examples, the fallback recovers 0.71 of the runtime extractor’s LangGraph edges—closely matching its 0.68 on mined LangGraph code—missing chiefly the conditional edges it cannot resolve statically; for ADK the runtime graph is far richer (16 vs. ~ 5 edges). Realising this higher-fidelity ceiling at scale would require executing untrusted repository code in a sandbox, which we do not attempt—and that gap between the faithful-but-unscalable and scalable-but-lossy extraction paths is exactly why extraction fidelity, not the check algorithms, is the binding constraint.

Defects are rare. Of 187 flagged defects, only 2 (1.1%, 95% CI [0.3, 3.8]%) are genuine bugs—both missing human gates—while 43% are extraction artifacts and 50% intentional design (Table 2). *Every one of the 72 structural flags*

is an extraction artifact (0 genuine; 95% CI [0, 5.1]%), and structural flags are entirely a LangGraph phenomenon: in the raw study they fire on 78% of LangGraph workflows but 0% of AutoGen, CrewAI, and ADK, whose extractors build a connected topology by construction and cannot emit a dead end or unreachable exit. The structural signal is thus an artifact of the one extractor that recovers real edges, not a property of the workflows.

Risk-aware gating. The human-gate check fires on 91% of workflows, but only 2 of 115 firings are genuine—the rest are intentional low-risk tasks. A risk-aware variant flags a workflow only when a *sensitive* tool (destructive/write/exec/communication/ financial, by a keyword lexicon) is reachable without a HUMAN gate, reusing the sound coverage check of Section . On the 930 mined workflows it fires on *zero*: none declares a sensitive tool, as they are read-only retrieval pipelines that need no review. But its recall is itself bounded by extraction fidelity—an in-body sensitive call with no declared binding (the `tool→llm` error above) is invisible—so a sound sensitivity signal depends on the same higher-fidelity extraction we identify as the binding constraint.

Controlled Evaluation and Monitor Selection

Controlled detection test. A curated corpus of 18 workflows spanning all four frameworks, seeded with known anti-patterns, confirms the checks fire when defects are genuinely present: Agentproof detects every injected structural defect and, with a risk-aware sensitive-tool set, flags 8 workflows whose sensitive path (e.g. `account_create`, `publish`, `auto_remediate`) is reachable without a human gate, versus 10 for the blunt human-presence check—declining to flag two workflows that lack a human node but perform no sensitive action. This corpus is a controlled test that the algorithms work, not evidence of prevalence. Verification is sub-second even for synthetic graphs of 5,000 nodes (far larger than any observed workflow); details, per-framework extractor accuracy on stubs, and a Promela/SMV modeling-effort comparison are in the supplementary material.

Monitor selection via static pruning. The graph $\times$ DFA product gives the static machinery a job that does not depend on defects existing. Because the product explores a superset of runtime paths, a policy whose product reaches no violation state can never fire at runtime on that workflow: its monitor is provably inert and need not be deployed. This is a sound *monitor-selection* step—deploy per workflow only the monitors that can actually fire. Across the curated corpus and 15 policies, 252 of 270 monitor instances (93.3%) are provably inert, a mean of 14 of 15 monitors skipped per workflow, so runtime enforcement evaluates only the remaining 6.7%; multi-tool nodes are first split into single-tool chains so the analysis never under-approximates a node’s events.

We report two honest limits. First, on today’s small, mostly linear workflows almost all inertness is *alphabet-level* (the policy’s atoms never appear); the reachability product proves only one further case, and its advantage over a plain symbol-set check grows with branching structure. Second, the fixed

policy vocabulary does not match real code—none of the 930 mined workflows invokes any of the 15 policies’ tools—so measuring pruning on real workflows requires instantiating policy templates against each workflow’s own tools, which we leave to future work. Static monitor selection is thus a sound, cheap optimization whose leverage today is predominantly alphabet-level, and it is the concrete bridge to runtime enforcement.

Related Work

Runtime agent safety. NeMo Guardrails (NVIDIA 2023), Guardrails AI (Guardrails AI 2023), LlamaGuard (Inan et al. 2023), and Rebuff (Protectai 2023) enforce safety at the point of LLM output or tool call, catching content-level violations only when the offending path runs. Agentproof is complementary: it verifies structural properties of the topology before deployment, and its monitor-selection layer (Section 4) decides which of these runtime monitors are even needed.

Model checking and workflow soundness. Classical model checkers—SPIN (Holzmann 1997), NuSMV (Cimatti et al. 2002), CBMC (Kroening and Tautschnig 2014)—verify the same class of properties but require manual translation into Promela/SMV/annotated C; Agentproof extracts the model automatically. Business-process soundness for Petri nets (van der Aalst 2011) and BPMN (Dijkman, Dumas, and Ouyang 2008) studies analogous reachability properties, but targets visual notations rather than the programmatic API objects agent frameworks expose. Runtime-verification frameworks such as JavaMOP (Jin et al. 2012) and three-valued LTL monitoring (Bauer, Leucker, and Schallhart 2011) inform our lightweight DFA monitors.

Agent safety and evaluation. Work on agent safety emphasizes alignment, capability evaluation, and governance (Anthropic 2023; METR 2024; Gabriel et al. 2024). Our contribution is orthogonal and empirical: the first base-rate measurement of a specific, checkable defect class in deployed workflows, and a diagnosis of what actually limits static verification for them.

Conclusion

Agent frameworks encode behavior as explicit workflow graphs, and AGENTPROOF turns that structure into an analyzable, cross-framework model with six structural checks, witness traces, and a safety-fragment-LTL policy DSL evaluated both statically (via a graph $\times$ DFA product) and at runtime. Used as a measurement instrument on 930 workflows from 253 repositories across four frameworks, it yields a deflationary but genuine finding: real workflows are mostly small static pipelines, and only 2 of 187 flagged defects are real bugs, the rest extraction artifacts or intentional design. Curated-benchmark defect rates do not stand in for prevalence.

The contribution survives the rarity of defects. First, a measurement instrument and the first real-world base rates for agent-workflow topology defects. Second, a quantified diagnosis: extraction fidelity, not the check algorithms, is the binding constraint—edge recall 0.64, collapsing to 0.37

on implicit-topology frameworks like ADK, with every triaged structural flag an extraction artifact. Third, a defect-independent use for the static machinery: the graph $\times$ DFA product soundly prunes provably-inert runtime monitors, making pre-deployment analysis a monitor-selection layer within a static-plus-runtime safety stack. We read the negative prevalence result not as a null result but as a redirection—toward higher-fidelity extraction and risk-aware policy specification, the direction we pursue in follow-up work on runtime shields.

Artifact. The tool, corpora, mining and validation pipeline, and all evaluation scripts will be released under the MIT license.

Ethical Statement

The study mines only *public* GitHub repositories, records exact provenance (repository at commit SHA plus file path), and redistributes only the derived graph abstractions and analysis, never third-party source. No personal data is processed. The intended impact is defensive—helping developers catch topology-level safety defects before deployment and reason about which runtime safeguards a workflow needs; the honest finding that such defects are currently rare should temper, not inflate, claims about static verification’s role in agent safety.

References

- Anthropic. 2023. Anthropic’s Responsible Scaling Policy. <https://www.anthropic.com/responsible-scaling-policy>. Accessed: 2026-03-01.
- Bauer, A.; Leucker, M.; and Schallhart, C. 2011. Runtime Verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology*, 20(4).
- Cimatti, A.; Clarke, E.; Giunchiglia, E.; et al. 2002. NuSMV 2: An OpenSource Tool for Symbolic Model Checking. In *International Conference on Computer Aided Verification (CAV)*. Springer.
- Clarke, E. M.; Grumberg, O.; and Peled, D. A. 1999. *Model Checking*. MIT Press.
- CrewAI. 2025. CrewAI documentation. <https://docs.crewai.com>. Accessed: 2026-02-28.
- Dijkman, R. M.; Dumas, M.; and Ouyang, C. 2008. Semantics and analysis of business process models in BPMN. *Information and Software Technology*, 50(12): 1281–1294.
- Gabriel, I.; et al. 2024. The Ethics of Advanced AI Assistants. *arXiv preprint arXiv:2404.16244*.
- Google. 2025. Google Agent Development Kit (ADK) documentation. <https://google.github.io/adk-docs/>. Accessed: 2026-02-28.
- Guardrails AI. 2023. Guardrails AI: Adding Guardrails to Large Language Models. <https://github.com/guardrails-ai/guardrails>. Accessed: 2026-03-01.
- Holzmann, G. J. 1997. The Model Checker SPIN. *IEEE Transactions on Software Engineering*, 23(5): 279–295.
- Inan, H.; Upasani, K.; Chi, J.; et al. 2023. LlamaGuard: LLM-based Input-Output Safeguard for Human-AI Conversations. *arXiv preprint arXiv:2312.06674*.

Jin, D.; Meredith, P. O.; Lee, C.; and Roşu, G. 2012. JavaMOP: Efficient Parametric Runtime Monitoring Framework. In *International Conference on Software Engineering (ICSE)*.

Kroening, D.; and Tautschnig, M. 2014. CBMC – C Bounded Model Checker. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer.

LangChain. 2025. LangGraph documentation. <https://docs.langchain.com/oss/python/langgraph/overview>. Accessed: 2026-02-28.

METR. 2024. METR: Model Evaluation and Threat Research. <https://metr.org>. Accessed: 2026-03-01.

NVIDIA. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications. <https://github.com/NVIDIA/NeMo-Guardrails>. Accessed: 2026-03-01.

Pnueli, A. 1977. The Temporal Logic of Programs. In *18th Annual Symposium on Foundations of Computer Science (FOCS)*.

Protectai. 2023. Rebuff: Self-hardening Prompt Injection Detector. <https://github.com/protectai/rebuff>. Accessed: 2026-03-01.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.

van der Aalst, W. M. P. 2011. *Process Mining: Discovery, Conformance and Enhancement of Business Processes*. Springer.

Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; et al. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *Conference on Language Modeling (COLM)*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

Reproducibility Checklist

Instructions for Authors:

This document outlines key aspects for assessing reproducibility. Please provide your input by editing this .tex file directly.

For each question (that applies), replace the “Type your response here” text with your answer.

Example: If a question appears as

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
Type your response here
```

you would change it to:

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
yes
```

Please make sure to:

- Replace ONLY the “Type your response here” text and nothing else.
- Use one of the options listed for that question (e.g., **yes**, **no**, **partial**, or **NA**).
- **Not** modify any other part of the \question command or any other lines in this document.

You can \input this .tex file right before \end{document} of your main file or compile it as a stand-alone document. Check the instructions on your conference’s website to see if you will be asked to provide this checklist with your paper or separately.

1. General Paper Structure

- 1.1. Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes/partial/no/NA) **yes**
- 1.2. Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results (yes/no) **yes**
- 1.3. Provides well-marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes/no) **yes**

2. Theoretical Contributions

- 2.1. Does this paper make theoretical contributions? (yes/no) **yes**

If yes, please address the following points:

- 2.2. All assumptions and restrictions are stated clearly and formally (yes/partial/no) **yes**
- 2.3. All novel claims are stated formally (e.g., in theorem statements) (yes/partial/no) **partial**
- 2.4. Proofs of all novel claims are included (yes/partial/no) **partial**
- 2.5. Proof sketches or intuitions are given for complex and/or novel results (yes/partial/no) **yes**
- 2.6. Appropriate citations to theoretical tools used are given (yes/partial/no) **yes**
- 2.7. All theoretical claims are demonstrated empirically to hold (yes/partial/no/NA) **yes**
- 2.8. All experimental code used to eliminate or disprove claims is included (yes/no/NA) **NA**

3. Dataset Usage

- 3.1. Does this paper rely on one or more datasets? (yes/no) [yes](#)

If yes, please address the following points:

- 3.2. A motivation is given for why the experiments are conducted on the selected datasets (yes/partial/no/NA) [yes](#)
- 3.3. All novel datasets introduced in this paper are included in a data appendix (yes/partial/no/NA) [partial](#)
- 3.4. All novel datasets introduced in this paper will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no/NA) [yes](#)
- 3.5. All datasets drawn from the existing literature (potentially including authors' own previously published work) are accompanied by appropriate citations (yes/no/NA) [NA](#)
- 3.6. All datasets drawn from the existing literature (potentially including authors' own previously published work) are publicly available (yes/partial/no/NA) [NA](#)
- 3.7. All datasets that are not publicly available are described in detail, with explanation why publicly available alternatives are not scientifically satisfying (yes/partial/no/NA) [NA](#)

4. Computational Experiments

- 4.1. Does this paper include computational experiments? (yes/no) [yes](#)

If yes, please address the following points:

- 4.2. This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting the final parameter setting (yes/partial/no/NA) [NA](#)
- 4.3. Any code required for pre-processing data is included in the appendix (yes/partial/no) [yes](#)
- 4.4. All source code required for conducting and analyzing the experiments is included in a code appendix (yes/partial/no) [yes](#)
- 4.5. All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no) [yes](#)
- 4.6. All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from (yes/partial/no) [partial](#)
- 4.7. If an algorithm depends on randomness, then the

method used for setting seeds is described in a way sufficient to allow replication of results (yes/partial/no/NA) [yes](#)

- 4.8. This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks (yes/partial/no) [partial](#)
- 4.9. This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics (yes/partial/no) [yes](#)
- 4.10. This paper states the number of algorithm runs used to compute each reported result (yes/no) [yes](#)
- 4.11. Analysis of experiments goes beyond single-dimensional summaries of performance (e.g., average; median) to include measures of variation, confidence, or other distributional information (yes/no) [yes](#)
- 4.12. The significance of any improvement or decrease in performance is judged using appropriate statistical tests (e.g., Wilcoxon signed-rank) (yes/partial/no) [NA](#)
- 4.13. This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments (yes/partial/no/NA) [yes](#)